

Android 15 Launcher 客制化指导手册

文档版本
发布日期

V1.0
2024-06-24

紫光展锐（上海）科技有限公司



版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档介绍 Android 15 Launcher 客制化方法，内容涵盖 UI 客制化、功能客制化。

读者对象

本文档主要适用 Android 开发人员。

缩略语

缩略语	英文全称	中文解释
GMS	Google Mobile Services	谷歌移动服务
GTS	GMS Test Suite	GMS 测试套件
RRO	Runtime Resource Overlay	运行资源覆盖
SRO	Static Resource Overlay	静态资源覆盖

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.0	2024-06-24	第一次正式发布

关键字

Launcher、UI、桌面布局、图标

目 录

1 UI 客制化	1
1.1 桌面布局	1
1.2 布局配置	2
1.2.1 布局配置客制化	3
1.2.2 显示大小对布局的影响	4
1.3 预置图标	5
1.3.1 预置图标类型	5
1.3.2 预置 workspace 图标	9
1.3.3 预置 Hotseat 图标	9
1.4 图标客制化	10
1.4.1 Hotseat 应用图标名称显示	10
1.4.2 隐藏图标	12
1.5 搜索框客制化	13
1.5.1 移除 SmartSpace 小部件	13
1.5.2 移除搜索框	14
1.5.3 搜索框可拖拽	14
1.5.4 替换搜索框	15
1.6 待机界面客制化	16
1.6.1 待机引导动画	16
1.6.2 待机界面上下阴影	16
1.7 资源 Overlay 客制化	17
2 功能客制化	19
2.1 桌面样式	19
2.2 Hotseat 图标自适应	21

图目录

图 1-1 基本布局	1
图 1-2 Hotseat 显示标题	11
图 1-3 移除 Smartspace 小部件	13
图 1-4 替换搜索框	15
图 1-5 RRO 预置示例	17
图 2-1 桌面模式设置界面	19
图 2-2 双层和单层桌面	20
图 2-3 Hotseat 图标自适应	21

表目录

表 1-1 Profile 文件中的 Launcher 属性	2
表 1-2 决定应用图标和名称大小的三组配置	4
表 1-3 appwidget 配置属性	6
表 1-4 favorite 配置属性	6
表 1-5 shortcut 配置属性	7
表 1-6 folder 配置属性	8
表 1-7 resolve 配置属性	9
表 2-1 桌面样式配置	20

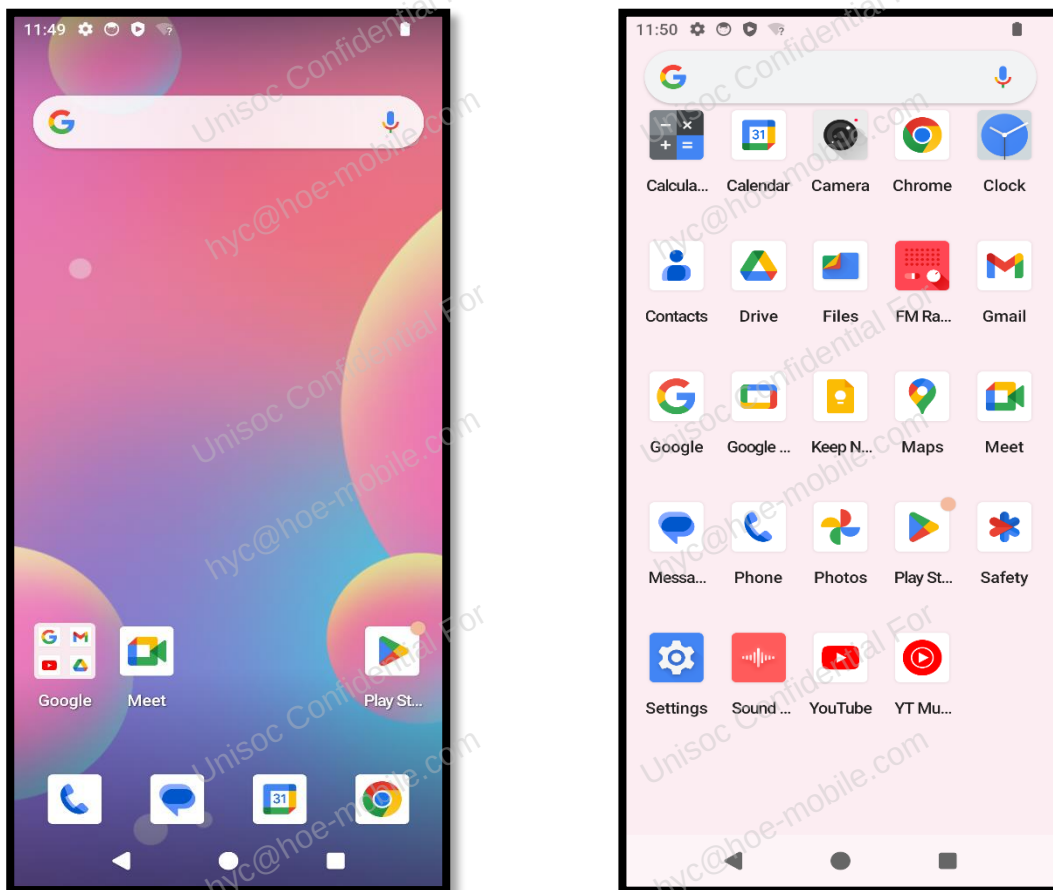
1 UI 客制化

Launcher 待机界面是用户使用频率最高的界面，待机界面的 UI 会影响用户对设备的第一印象。因此，为了增加产品竞争力，通常需要对桌面 UI 进行客制化。本章会逐一介绍常用的桌面 UI 客制化方法和注意事项。

1.1 桌面布局

图 1-1 是 Launcher3 的基本布局，主要分为 Workspace（左图）和 All Apps（右图）两个界面。

图1-1 基本布局



1.2 布局配置

布局配置文件路径：

- Go 版本：Launcher3\go\res\xml\device_profiles.xml
- 非 Go 版本：Launcher3\res\xml\device_profiles.xml

配置文件中定义了不同分辨率对应的 `grid-option` 和 `display-option` 标签，Launcher 根据设备分辨率自适应最佳 UI 配置，通常情况下无需调整。如需客制化，请参考 [1.2.1 布局配置客制化](#)。

下面以非 Go 版本的 profile 文件内容为例，说明各属性的含义，具体参见[表 1-1](#)。

```
<profiles xmlns:launcher="http://schemas.android.com/apk/res-auto" >

  <grid-option
    launcher:name="4_by_4"
    launcher:numRows="4"
    launcher:numColumns="4"
    launcher:numFolderRows="3"
    launcher:numFolderColumns="4"
    launcher:numHotseatIcons="4"
    launcher:numExtendedHotseatIcons="6"
    launcher:dbFile="launcher_4_by_4.db"
    launcher:inlineNavButtonsEndSpacing="@dimen/taskbar_button_margin_split"
    launcher:defaultLayoutId="@xml/default_workspace_4x4"
    launcher:deviceCategory="phone|multi_display" >

    <display-option
      launcher:name="Short Stubby"
      launcher:minWidthDps="275"
      launcher:minHeightDps="420"
      launcher:iconImageSize="48"
      launcher:iconTextSize="13.0"
      launcher:canBeDefault="true" />

  </grid-option>

</profiles>
```

一种`<grid-option>`和一种`<display-option>`为一组，对应一种桌面布局。

- `grid-option` 决定桌面应用图标行列数、文件夹内图标行列数、Hotseat 区域图标数目、数据库名称以及桌面预置图标位置。
- `display-option` 决定桌面应用图标和应用名称的大小。

表1-1 Profile 文件中的 Launcher 属性

属性	说明
<code><grid-option></code>	网格配置项 <code>name</code>：名称 <code>numRows</code>：待机界面应用行数 <code>numColumns</code>：待机界面和应用抽屉应用列数

属性	说明
	- numFolderRows: 文件夹中应用行数 - numFolderColumns: 文件夹中应用列数 - numHotseatIcons: Hotseat 应用图标数量 - numExtendedHotseatIcons: Hotseat 可扩展的图标数量 - dbFile: 此 grid-option 下使用的数据库名 - inlineNavButtonsEndSpacing: 行内导航按钮结束端与行边界的间距 - defaultLayoutId: 默认布局文件名 - deviceCategory: 设备类型
<display-option>	显示属性配置项 - name: 名称 - minWidthDps: 应用被允许的最小宽度，单位 dp - minHeightDps: 应用被允许的最小高度，单位 dp - iconImageSize: 图标大小 - iconTextSize: 字体大小 - canBeDefault: 是否可以作为默认配置

1.2.1 布局配置客制化

获取当前设备使用的配置信息

设备在待机界面时，在 PC 终端执行如下命令：

- Windows 环境

```
$ adb shell dumpsys activity com.android.launcher3.uioverrides.QuickstepLauncher | findstr DeviceProfile
```

- Ubuntu 环境

```
$ adb shell dumpsys activity com.android.launcher3.uioverrides.QuickstepLauncher | grep DeviceProfile
```

输出结果如下：

```
$ DeviceProfile: IDP [mGridName:4_by_4 mDisplayOptionName: <4_by_4, Nexus 5>&<4_by_4, Nexus 4>&<4_by_4, Nexus S> numRows:4 numColumns:4 numHotseatIcons:5 numFolderRows:3 numFolderColumns:4 iconSize:53.89797 i conShapePath:M50,0L92,0C96.42,0 100,4.58 100 8L100,92C100, 96.42 96.42 100 92 100L8 100C4.58, 100 0 96.42 0 92L0 8 C 0 4.42 4.42 0 8 0L50 0Z landscapeIconSize:5 3.89797 iconTextSize:13.0]
```

由于不同 Launcher 形态使用不同 Activity，上面红色部分需要根据具体使用的 Activity 进行替换。不同 Activity 对应的内容如下：

- Launcher3: com.android.launcher3.Launcher
- QuickStep: com.android.launcher3.uioverrides.QuickstepLauncher

- SearchLauncher: com.android.searchlauncher.SearchLauncher

例如，Windows 环境下，Launcher3 形态，替换后为：

```
$ adb shell dumpsys activity com.android.launcher3.Launcher | findstr DeviceProfile
```

说明

搭配了 QuickStep、SearchLauncher（Android 15 GMS 版本）形态的版本中，上述命令中的类名需要被替换才能生效，否则得到的结果为空。

根据配置信息客制化

以获取当前设备使用的配置信息输出的 Log 为例：

- “mGridName:4_by_4”说明当前桌面布局由 profile 文件中 Launcher name 为 4_by_4 的 grid-option 标签指定的。找到该 grid-option 标签项，可通过其下的各个 Launcher 属性自定义桌面图标行列数、文件夹内图标行列数、Hotseat 区域图标个数以及桌面默认图标及其位置，Launcher 属性具体含义见表 1-1。
- “mDisplayOptionName:<4_by_4, Nexus 5>&<4_by_4,Nexus 4>&<4_by_4,Nexus S>”说明当前桌面应用图标和应用名称大小由表 1-2 中的三组配置共同决定。调整应用图标或者应用名称的大小，只需分别微调三组配置中的 iconImageSize 和 iconTextSize。

表1-2 决定应用图标和名称大小的三组配置

Grid 名称	Display 名称	定义应用图标和应用名称大小
4_by_4	Nexus 5	Launcher:iconImageSize="48" Launcher:iconTextSize="13.0"
4_by_4	Nexus 4	Launcher:iconImageSize="54" Launcher:iconTextSize="13.0"
4_by_4	Nexus S	Launcher:iconImageSize="54" Launcher:iconTextSize="13.0"

设备配置信息中的 mDisplayOptionName 打印为 “mDisplayOptionName:Nexus 5” 这种形式时，说明当前桌面布局由 grid name 为 4_by_4 且 display name 为 Nexus 5 的这组配置决定。此时只需更改这组配置中的 iconImageSize 和 iconTextSize 即可客制化桌面应用图标和应用名称大小。

1.2.2 显示大小对布局的影响

在“系统设置>显示>显示大小”，可以改变设备的显示大小。每次修改显示大小后 Launcher 会重新适配布局配置。由于各布局配置的参数不同，修改显示大小后，可能会引起桌面行列数等布局发生变化。

说明

- 如果期望修改显示大小后桌面布局不发生变化，可以仿照 Go 版本配置，只保留一套 profile 中的 grid-option，但会导致桌面网格功能失效。
- 桌面网格功能开启时，更改显示大小不影响桌面行列数。

1.3 预置图标

设备首次启动完成后，待机界面已经预置了部分图标。配置文件会被 Partner 机制替换，建议先检查是否存在 Partner App。

1. 查看 Launcher 启动时的 Log，存在如下信息，表示存在 Partner App。

D\UNISOC-LAUNCHER3: **Launcher.Partner**, packageName is:com.google.android.gmsintegration

2. 根据包名查找对应配置文件。

以 GMS（Google Mobile Services，谷歌移动服务）版本为例，预置图标的配置文件路径如下：

vendor\partner_gms\unisoc_conf\apps\GmsSampleIntegration_unisoc\res_XXX\xml\partner_default_layout.xml（res_XXX 请根据项目配置情况确认）

以上为 UNISOC 解耦配置路径，若直接使用 Google GMS 配置，则修改路径为：

vendor\partner_gms\apps\GmsSampleIntegration_unisoc\res_XXX\xml\partner_default_layout.xml

默认图标在表 1-1 中的 defaultLayoutId 进行配置，对应如下文件：

Launcher3\res\xml\default_workspace_XXX.xml

如果修改此文件无效，需要考虑该文件是否被 Launcher Partner 机制替换。

本章以不存在 Partner App 为例，介绍预置图标的修改。如果存在 Partner App，需在 Partner App 对应的配置文件中进行相应修改。

1.3.1 预置图标类型

预置图标在上述 default_workspace_XXX 配置文件中添加，共有如下 5 种类型：

- appwidget（待机界面小部件）
- favorite（待机界面应用图标）
- shortcut（待机界面快捷方式图标）
- folder（待机界面文件夹）
- resolve（待机界面图标）

1.3.1.1 appwidget

待机界面小部件格式如下：

```
<appwidget
    launcher:screen="0"
    launcher:x="0"
    launcher:y="1"
    launcher:spanX="4"
    launcher:spanY="2"
    launcher:packageName="com.android.deskclock"
    launcher:className="com.android.alarmclock.DigitalAppWidgetProvider" />
```

表1-3 appwidget 配置属性

名称	定义
screen	页面位置，第几页
x	X 坐标位置
y	Y 坐标位置
spanX	Widget 宽度占用格子数
spanY	Widget 高度占用格子数
packageName	Widget 包名
className	Widget 类名

注意

其他图标或 widget 的预置位置不能与本 widget 所预置的区域冲突，否则会导致加载失败。

1.3.1.2 favorite

待机界面应用图标格式如下：

```
<favorite
  launcher:screen="0"
  launcher:x="2"
  launcher:y="3"
  launcher:packageName="com.android.settings"
  launcher:className="com.android.settings.Settings" />
```

表1-4 favorite 配置属性

名称	定义
screen	页面位置，第几页
x	X 坐标位置
y	Y 坐标位置
packageName	包名
className	类名

说明

favorite 预置，一般预置带有 Launcher 属性的应用图标。

1.3.1.3 shortcut

待机界面快捷方式图标格式如下：

```
<shortcut
    launcher:icon="@drawable/app_icon"
    launcher:title="@string/app_name"
    launcher:uri="http://www.baidu.com/"
    launcher:screen="0"
    launcher:x="0"
    launcher:y="0" />
```

表1-5 shortcut 配置属性

名称	定义
icon	图标
title	名称
uri	uri 类型的地址
screen	页面位置，第几页
x	X 坐标位置
y	Y 坐标位置

shortcut 预置，一般预置的是网址书签、应用中某个 Activity（不带 Launcher 属性）、打开某个特殊文件等快捷方式，可以自定义其 icon、title，通常建议使用“launcher:uri”标签。

例如，打开预置在某个目录的视频文件，可以使用如下 uri 格式：

```
launcher:uri="file:///storage/emulated/0/Movies/1.mp4#Intent;type=video/*;
action=android.intent.action.VIEW;category=android.intent.category.BROWSABLE;end"
```

若需指定系统默认播放器，增加如下内容：

```
component=com.android.gallery3d/.app.MovieActivity;
```

若需选择播放器，删除增加的内容即可。

说明

使用 shortcut 标签，必须设置 title，否则预置会失效。

1.3.1.4 folder

待机界面文件夹格式如下：

```
<folder
    launcher:title="@string/folder_name"
    launcher:screen="0"
```



```
launcher:x="0"
launcher:y="3">
<favorite
    launcher:packageName="com.android.settings"
    launcher:className="com.android.settings.Settings"
    launcher:x="0"/>
<favorite
    launcher:packageName="com.android.settings"
    launcher:className="com.android.settings.Settings"
    launcher:x="1"/>
</folder>
```

表1-6 folder 配置属性

名称	定义
title	名称
screen	页面位置，第几页。
x	X 坐标位置
y	Y 坐标位置
favorite	文件夹中预置的图标，详见表 1-4，此时 x 代表文件夹中的排序

说明

folder 预置，不能包含 appwidget 类型。

1.3.1.5 resolve

待机界面图标格式如下：

```
<resolve
    launcher:container="-101"
    launcher:screen="1"
    launcher:x="1"
    launcher:y="0" >
<favorite
    launcher:uri="#Intent;action=android.intent.action.MAIN;category=android.intent.category.APP_ME
SSAGING;end" />
<favorite launcher:uri="sms:" />
<favorite launcher:uri="smsto:" />
<favorite launcher:uri="mms:" />
<favorite launcher:uri="mmsto:" />
```

</resolve>

表1-7 resolve 配置属性

名称	定义
container	图标的容器 • -101 代表 Hotseat • -100 代表 Workspace
screen	页面位置，也是 Hotseat 自左到右的顺序位置
x	X 坐标位置，也是 Hotseat 自左到右的顺序位置，与 screen 一致
y	Y 坐标位置，由于 Hotseat 只有一行，故默认一直是 0
favorite	一般指地址，详见 表 1-4

说明

resolve 预置，一般预置需要自适应的应用图标，favorite 中配置 uri。

1.3.2 预置 workspace 图标

可以直接修改如下 workspace 待机界面图标布局文件。

packages/apps/Launcher3/res/xml/default_workspace_XXX.xml

<favorite

launcher:screen="0"

launcher:x="-1"

launcher:y="-1"

launcher:packageName="com.android.settings"

launcher:className="com.android.settings.Settings" />

说明

其中 x 轴或者 y 轴位置若是“-1”，表示待机界面 x 轴或者 y 轴的最后一个位置。

如果存在 Partner App，请在 Partner App 中修改待机界面默认布局文件。

1.3.3 预置 Hotseat 图标

Hotseat 图标位于待机界面最下面的一行，通常预置常驻应用，在如下文件中配置：

Launcher3/res/xml/default_workspace_XXX.xml

<resolve

launcher:container="-101"

launcher:screen="1"

launcher:x="1"

launcher:y="0" >

```
<favorite
    launcher:uri="#Intent;action=android.intent.action.MAIN;category=android.intent.category.APP_ME
SSAGING;end" />
<favorite launcher:uri="sms:" />
<favorite launcher:uri="smsto:" />
<favorite launcher:uri="mms:" />
<favorite launcher:uri="mmsto:" />
</resolve>
```

预置 Hotseat 图标使用 resolve 标签和 uri 属性，可以自适应。例如，预置了第三方短信应用而没有使用原生短信应用，则该位置会根据 resolve 标签和 uri 属性解析出第三方短信应用，进而填充 Hotseat 图标。

1.4 图标客制化

对图标进行客制化，请参考 [1.2 布局配置](#)，确认设备使用的 grid-option 标签，然后调整属性即可。其他复杂的情况，则需要调整代码来实现，例如双行显示、Hotseat 应用图标名称显示和隐藏图标等。

1.4.1 Hotseat 应用图标名称显示

Hotseat 上的图标为最常用的图标，Google 设计为通过图标辨识应用，隐藏了 Hotseat 图标的 title。非特殊情况，不建议显示该 title，因为显示 Hotseat title 会导致待机界面可用区域变小，甚至会导致某些 Widget 显示不全，请慎重修改。

Hotseat 区域，应用图标名称默认隐藏，若需显示，请按照如下步骤进行修改：

步骤 1 显示 Hotseat 应用图标名称。

```
Launcher3\src\com\android\launcher3\CellLayout.java
// Hotseat icons - remove text
if (child instanceof BubbleTextView) {
    BubbleTextView bubbleChild = (BubbleTextView) child;
-    bubbleChild.setTextVisibility(mContainerType != HOTSEAT);
+    bubbleChild.setTextVisibility(true);
}
```

步骤 2 显示 Hotseat 区域文件夹图标名称。

```
Launcher3\src\com\android\launcher3\WorkspaceLayoutManager.java
// Hide folder title in the hotseat
if (child instanceof FolderIcon) {
-    ((FolderIcon) child).setTextVisible(false);
+    ((FolderIcon) child).setTextVisible(true);
}
```

步骤 3 拖拽动画结束后，显示 Hotseat 区域应用名称。

```
Launcher3\src\com\android\launcher3\BubbleTextView.java
```

```
public boolean shouldTextBeVisible() {  
-    // Text should be visible everywhere but the hotseat.  
-    Object tag = getParent() instanceof FolderIcon ? ((View) getParent()).getTag() : getTag();  
-    ItemInfo info = tag instanceof ItemInfo ? (ItemInfo) tag : null;  
-    return info == null || info.container != LauncherSettings.Favorites.CONTAINER_HOTSEAT;  
+    return true;  
}
```

步骤 4 调整 Hotseat 图标大小。

若 Hotseat 图标名称显示不全，需要调整 Hotseat 大小，使图标名称显示完全。请根据实际情况调整。

Launcher3\res\values\dimens.xml

```
-    <dimen name="dynamic_grid_hotseat_extra_vertical_size">34dp</dimen>  
+    <dimen name="dynamic_grid_hotseat_extra_vertical_size">48dp</dimen>
```

修改前后的效果如图 1-2 所示。

图1-2 Hotseat 显示标题



步骤 5 调整 Hotseat 图标顺序。

- GMS 版本
vendor\partner_gms\unisoc_conf\apps\GmsSampleIntegration_unisoc\res_dhs_full\xml\partner_default_layout.xml
- GMS-GO 版本
vendor\partner_gms\unisoc_conf\apps\GmsSampleIntegration_unisoc\res_dhs_go_2gb\xml\partner_default_layout.xml

-----结束

1.4.2 隐藏图标

在 Launcher 中隐藏某个应用图标及其 Widget，以达到预期效果。目前，Launcher 已经预留了隐藏某个应用的接口类：

Launcher3\src\com\android\launcher3\AppFilter.java

步骤 1 新增一个类继承于接口类。

例如 Launcher3\src\com\android\launcher3\CustomAppFilter.java
package com.android.launcher3;

import android.content.ComponentName;

import android.content.Context;

public class CustomAppFilter extends AppFilter {

public CustomAppFilter(Context context) {

}

@Override

public boolean shouldShowApp(ComponentName app) {

// 实现此方法即可过滤相应的应用图标

if (app.getPackageName().equals("com.android.calendar")) {

return false;

}

return true;

}

}

步骤 2 在配置文件中配置该类。

Launcher3\res\values\config.xml

- <string-array name="filtered_components"></string-array >

+ <string-array name="filtered_components">com.android.launcher3.CustomAppFilter</string-array>

该子类为隐式启动，其他地方没有调用，在编译过程中会被忽略。需在 Launcher3\proguard.flags 中添加如下内容，避免被忽略。

+ -keep class com.android.launcher3.CustomAppFilter {

+ *;

+ }

----结束

1.5 搜索框客制化

首页搜索框为 Google 的强制要求，非特殊原因不建议移除、修改，否则会导致 Google GTS（GMS Test Suite，GMS 测试套件）测试无法通过。

1.5.1 移除 Smartspace 小部件

SearchLauncherQuickStep 上方有个固定的 Smartspace 小部件图 1-3（左图），替换了 Launcher3 中的搜索框小部件，无法拖动、移除。

步骤 1 移除 Smartspace 小部件。

Launcher3\src\com\android\launcher3\config\FeatureFlags.java

```
private FeatureFlags() {}  
  
- public static final boolean QSB_ON_FIRST_SCREEN = true;  
+ public static final boolean QSB_ON_FIRST_SCREEN = false;
```

步骤 2 移除“主屏幕设置”中的“At a glance”。

vendor\partner_gms\apps\SearchLauncher\res\xml\launcher_preferences.xml

```
- <androidx.preference.PreferenceScreen  
-     android:key="pref_smartspace"  
-     android:persistent="false"  
-     android:summary="@string/smartspace_preferences_in_settings_desc"  
-     android:title="@string/smartspace_preferences_in_settings"/>
```

图1-3 移除 Smartspace 小部件



-----结束

1.5.2 移除搜索框

移除搜索框分为以下两种情况：

- 在 GMS 版本的 SearchLauncherQuickStep 中移除
- 在非 GMS 版本的 Launcher3 中移除

在 GMS 版本的 SearchLauncherQuickStep 中移除

删除如下文件，移除后主菜单中的搜索框位置会有稍许变动。

vendor\partner_gms\apps\SearchLauncher 目录下：

```
deleted:    quickstep\res\layout\search_container_all_apps.xml
deleted:    quickstep\res\values\dimens.xml
deleted:    quickstep\src\com\android\searchlauncher\HotseatQsbWidget.java
```

在非 GMS 版本 Launcher3 中移除

这种情况下移除比较简单，只需要在 Launcher3\src\com\android\launcher3\config\BaseFlags.java 中，进行如下修改：

```
// Enable moving the QSB on the 0th screen of the workspace
- public static final boolean QSB_ON_FIRST_SCREEN = true;
+ public static final boolean QSB_ON_FIRST_SCREEN = false;
```

1.5.3 搜索框可拖拽

目前待机界面默认搜索框是固定显示的，无法拖拽、删除。若需要搜索框可拖拽、删除，需要做如下修改。

注意

此修改可能会导致 Google GTS 测试无法通过。

步骤 1 移除固定的搜索框小部件。

方法可以参考 [1.5.1 移除 Smartspace 小部件](#) 和 [1.5.2 移除搜索框](#)，将固定的元素移除。

步骤 2 添加可以拖动的搜索小部件，确保小部件的包名、类名正确。

小部件的包名、类名可以通过拖动小部件到桌面，然后导出并查看 Launcher 数据库获得。对于无 Partner App 和有 Partner App 的版本需要修改不同的.xml 文件。

- 无 Partner App 的版本，定位到 Launcher3\res\xml\default_workspace_XX.xml

```
<appwidget
    launcher:screen="0"
    launcher:x="0"
    launcher:y="0"
    launcher:spanX="4"
    launcher:spanY="1"
```

```
launcher:packageName="com.google.android.googlequicksearchbox"  
launcher:className="com.google.android.googlequicksearchbox.SearchWidgetProvider"/>
```

- 有 Partner App 的版本，定位到 Partner App 下的\xml\partner_default_layout.xml

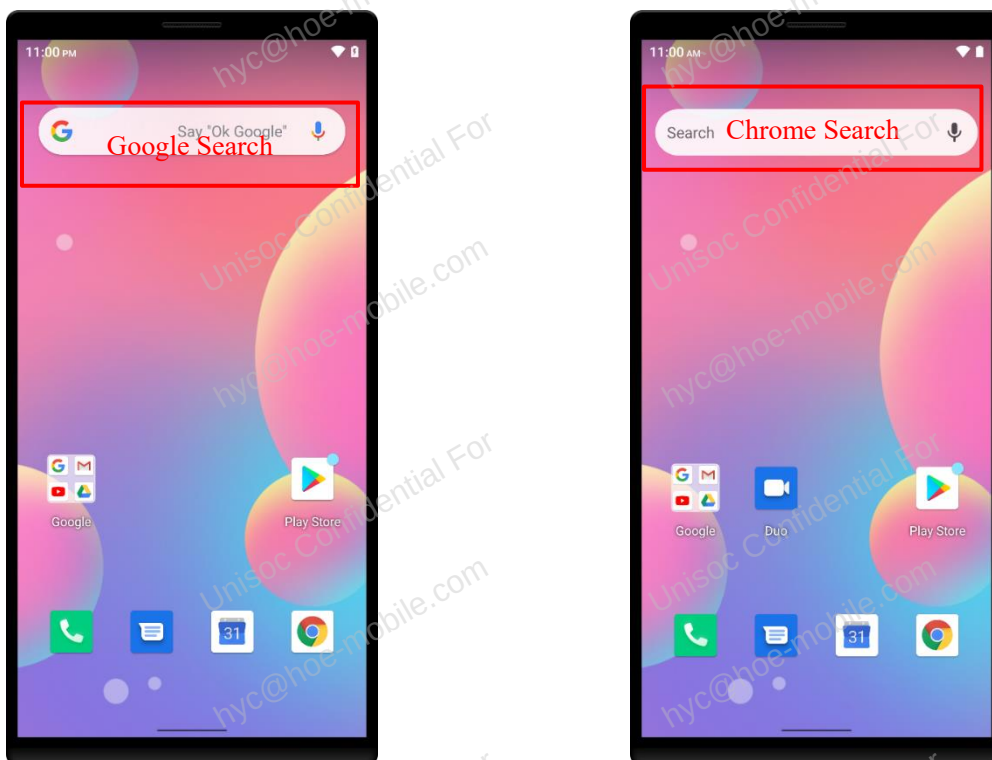
```
<appwidget  
    screen="0"  
    x="0"  
    y="0"  
    spanX="4"  
    spanY="1"  
    packageName="com.google.android.googlequicksearchbox"  
    className="com.google.android.googlequicksearchbox.SearchWidgetProvider"/>
```

----结束

1.5.4 替换搜索框

以替换为 Chrome 搜索框为例，修改前后效果如图 1-4 所示。

图1-4 替换搜索框



方案修改如下：

vendor\sprd\platform\packages\apps\Launcher3\res_unisoc\values\config_ext.xml

```
+ <!--add for use custom search widget-->
+ <string name="custom_search_component" translatable="false">
com.android.chrome/org.chromium.chrome.browser.searchwidget.SearchWidgetProvider</string>
```

若需要修改为其他搜索框，请自行调整 `custom_search_component`。

说明

配置 `config` 开关，可以使用 SRO（Static Resource Overlay，静态资源替换）方式开启此该功能，也可以通过 RRO（Runtime Resource Overlay，运行资源覆盖）的方式开启。注意 Chrome 浏览器的包名、类名必须正确。

1.6 待机界面客制化

待机界面是指开机解锁后或者按 Home 键后进入的界面。

1.6.1 待机引导动画

首次开机后，Hotseat 区域会连续向上跳动六次，提示用户可以通过向上滑动进入应用抽屉界面，这是一个进入应用抽屉引导动画。一旦用户进入应用抽屉后，该动画就不会再播放。

该动画在如下文件中定义，若需修改效果，请自行在该文件中修改。

`packages/apps/Launcher3/res/animator/discovery_bounce.xml`

若需关闭此动画，请定位到如下文件：

`packages/apps/Launcher3/src/com/android/launcher3/allapps/DiscoveryBounce.java`

```
+ private static final boolean HIDE_DISCOVERY_BOUNCE = true;
+
public static void showForHomeIfNeeded(Launcher launcher) {
+     if (HIDE_DISCOVERY_BOUNCE) return; //直接return即可关闭此动画
    showForHomeIfNeeded(launcher, true);
}
public static void showForOverviewIfNeeded(Launcher launcher,
                                           PagedOrientationHandler orientationHandler) {
+     if (HIDE_DISCOVERY_BOUNCE) return;
    showForOverviewIfNeeded(launcher, true, orientationHandler);
}
```

1.6.2 待机界面上下阴影

待机界面从状态栏向下有一个渐变的阴影效果，若需去除此效果，在如下文件中修改：

`Launcher3/res/values/styles.xml`

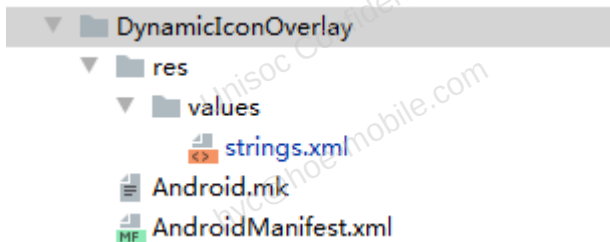
```
- <item name="workspaceStatusBarScrim">@drawable/workspace_bg</item>
+ <item name="workspaceStatusBarScrim">@null</item>
```

1.7 资源 Overlay 客制化

Launcher 存在多种形态，在编译时会引用不同目录下的 res 文件（具体引用可以查看 Android.mk）。使用 SRO 时，需要考虑 SRO 文件是否被引用，否则会导致 SRO 不生效。建议使用 Google 推荐的 RRO 方式做 Overlay。

本节只简单实现一个 RRO APK 的模板，更详细介绍请参考 Google 相关文档。
<https://source.android.google.cn/devices/architecture/rros>

图1-5 RRO 预置示例



步骤 1 strings.xml 中包含需要替换的字符串资源，布局文件不能被替换。

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Launcher3Overlay</string>
</resources>
```

步骤 2 Android.mk 实现将 RRO 命令行编译成 APK 应用。

```
#####
# Various Dynamic Icon targets
LOCAL_PATH:= $(my-dir)
# NativeDynamicIconOverlay
include $(CLEAR_VARS)
LOCAL_PACKAGE_NAME := NativeDynamicIconOverlay
LOCAL_MODULE_OWNER := unisoc
LOCAL_MODULE_TAGS := optional
LOCAL_SYSTEM_EXT_MODULE := true
LOCAL_CERTIFICATE := platform
LOCAL_RESOURCE_DIR := $(LOCAL_PATH)/res
LOCAL_SDK_VERSION := current
include $(BUILD_RRO_PACKAGE)
```

步骤 3 AndroidManifest.xml 定义了应用的包名、替换目标包名、优先级（取值范围[-1000, 1000]，值越大优先级越高，默认值为 0）、版本、是否是 Static 类型。

```
<manifest xmlns:android=http://schemas.android.com/apk/res/android
```

```
package="com.android.overlay.dynamiciconconfig"  
android:versionCode="1"  
android:versionName="1.0">  
<overlay android:targetPackage="com.android.launcher3"  
    android:priority="0"  
    android:isStatic="true" />  
</manifest>
```

步骤 4 NativeDynamicIconOverlay 在对应项目中参与编译。

```
PRODUCT_PACKAGES += NativeDynamicIconOverlay
```

----结束

2 功能客制化

本章节主要对 Launcher 的常用功能进行介绍。

PROP 属性：ro.launcher.XXX。

PROP 属性的默认配置及修改方法请参考默认配置，路径如下：

`\device\sprd\sys_mpool\module\system_ext\app\launcher\`

2.1 桌面样式

功能简介

桌面样式主要有单层桌面、双层桌面两种（如图 2-2）。动态切换开启后，可以在“主屏幕设置”中的“桌面样式”选择样式（如图 2-1）。

图2-1 桌面模式设置界面

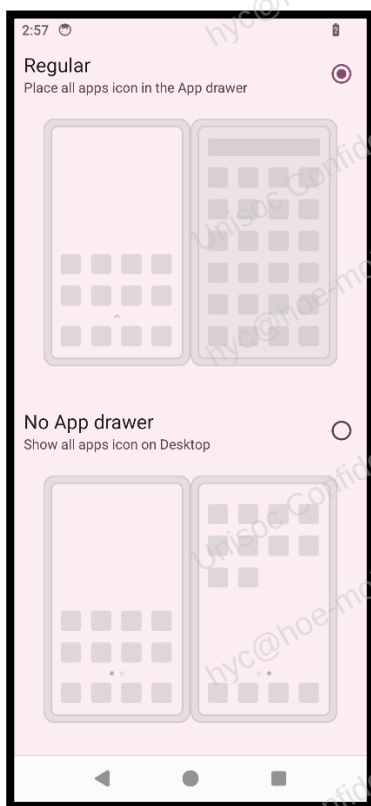
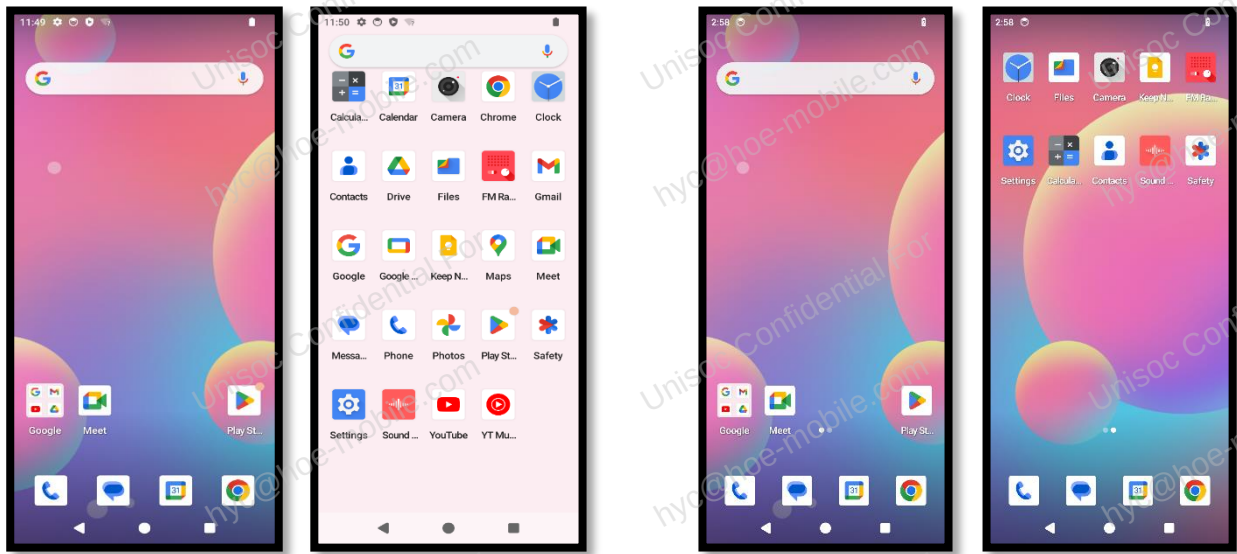


图2-2 双层和单层桌面



功能开关

桌面样式切换功能通过如下 prop 属性控制，可根据项目需求开启或者关闭。

prop 属性：ro.launcher.multimode

功能默认状态

- Android 15 非 Go 版本默认开启。
- Android 15 Go 版本默认关闭。

功能配置

表2-1 桌面样式配置

目标模式		ro.launcher.multimode	show_home_screen_style_settings	default_home_screen_style
仅双层桌面		false	false	无需配置
仅单层桌面		true	false	single
支持动态切换	默认单层	true	true	single
	默认双层	true	true	dual

主要代码目录：

Vendor\sprd\platform\packages\apps\Launcher3\src\com\sprd\ext\multimode

功能自定义调整:

Vendor\sprd\platform\packages\apps\Launcher3\res_unisoc\values\config_ext.xml

- 配置在“主屏幕设置”中是否显示“桌面样式”选项。

```
<bool name="show_home_screen_style_settings">true</bool>
```

- 配置默认样式，默认值请使用 home_screen_style_values 定义的 value 值，若配置错误，则默认为双层桌面。

```
<string name="default_home_screen_style" translatable="false">dual</string>
```

说明

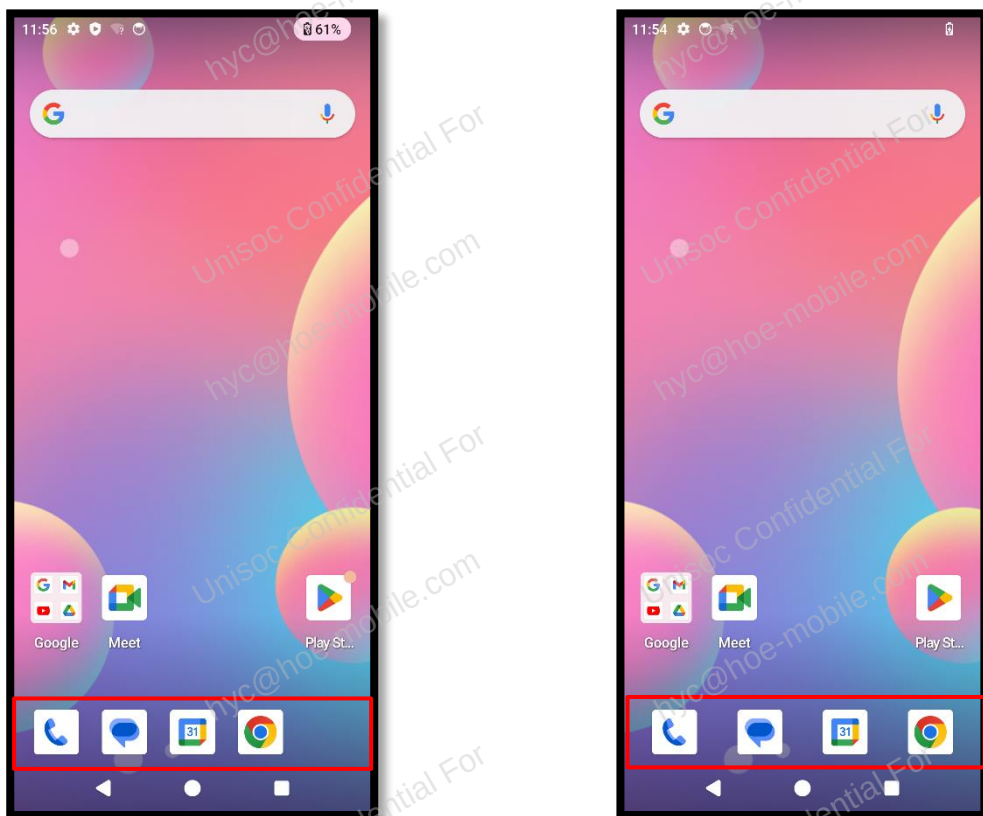
home_screen_style_values 数组在 Launcher3\res_unisoc\values\arrays_ext.xml 中定义。

2.2 Hotseat 图标自适应

功能简介

Hotseat 图标个数发生变化后，Hotseat 图标自适应功能可以自动平均分配 Hotseat 空间，重新对 Hotseat 布局，如图 2-3 所示。

图2-3 Hotseat 图标自适应



功能开关

Hotseat 图标自适应功能通过如下 prop 属性控制，可根据项目需求开启或者关闭功能。

prop 属性：ro.launcher.hs.adaptive

功能默认状态

- Android 15 非 Go 版本默认开启。
- Android 15 Go 版本默认关闭。

功能配置

主要代码目录：

Vendor\sprd\platform\packages\apps\src\com\sprd\ext\grid\HotseatController.java